MONITORING PRIMER · AZURE + DATABRICKS

Monitoring Databricks with Azure Monitor and Log Analytics

Azure Monitor collects signals from every Azure service. **Log Analytics** is where those signals are stored and queried. Connect Databricks to both, and the platform team sees failures, costs and security events alongside the rest of Azure.

> LEARNING OBJECTIVE
>
> Describe how Azure Databricks signals reach Azure Monitor, how Log Analytics stores and queries them, and how to turn them into alerts and dashboards.

01 Why: what goes wrong without monitoring

Why monitor the platform?

A data platform fails quietly. A nightly job stops, a cluster runs all weekend, a permission is changed. Users notice days later, when a report is wrong or the invoice arrives. Monitoring moves that moment to minutes after it happens.

RISK	SIGNAL THAT CATCHES IT
A nightly pipeline fails and reports show stale numbers	**Job run events.** Alert on runFailed within minutes.
A large cluster runs all weekend	**Cluster events + billing.** Flag long-running clusters and DBU spikes.
Someone grants themselves access to sensitive data	**Audit events.** Alert the security team on permission changes.
Jobs slowly take longer until they miss their deadline	**Run duration trends.** Track against the SLA before it breaks.
Workspace or network settings are changed outside change control	**Activity log.** Record and alert on changes to Azure resources.

Why send it to Azure Monitor?

Databricks has its own monitoring pages and system tables. Azure Monitor adds what an operations team needs across the whole estate.

1. **One place for all of Azure**
   Databricks events sit next to storage, Key Vault, network and Entra ID logs.
2. **Real-time alerting**
   Log search alerts evaluate every few minutes and page the on-call team.
3. **Retention you control**
   Keep logs for as long as audit rules require, independent of Databricks.
4. **Separation of duties**
   The operations or security team owns the logs; workspace admins can't change them.
5. **Existing tools**
   Sentinel, ServiceNow, Grafana and Power BI already connect to Log Analytics.

02 The map: signals, routes and destinations

Databricks produces several kinds of signal, and each reaches Azure Monitor by a different route. Some never reach it and are best read from Databricks system tables.

01 Databricks diagnostic logs — Audit events per service

ROUTE

Diagnostic setting on the workspace

Azure Monitor receives Databricks audit events by category (jobs, clusters, notebooks, Unity Catalog and more) within minutes.

`Set up in: Azure portal → workspace → Diagnostic settings`

LANDS IN

- `DatabricksJobs`
- `DatabricksClusters`
- `DatabricksUnityCatalog`

One table per category you enable.

USE IT FOR

- Job failure and cluster activity alerts
- Security investigations and Sentinel rules
- Long-term audit retention

Gap: Events, not metrics: no CPU, memory or Spark stage timings.

02 Azure Activity log — Changes to the Azure resource

ROUTE

Automatic; export with a subscription diagnostic setting

Records operations on the workspace resource itself: create, update, delete, role assignments, network changes.

`Set up in: Subscription → Activity log → Export`

LANDS IN

- `AzureActivity`

Kept 90 days in Azure by default; export for longer.

USE IT FOR

- Who changed or deleted the workspace
- Alerts on RBAC and network changes
- Resource Health notifications

Gap: Nothing that happens inside the workspace.

03 Compute metrics

CPU, memory, disk of cluster VMs

ROUTE

Databricks compute metrics UI; Azure VM platform metrics

Databricks shows per-cluster hardware and Spark metrics in the compute page. Cluster VMs also emit Azure platform metrics in the managed resource group, but they are short-lived and hard to tie to a cluster.

Set up in: Built in; optional Azure Monitor Agent via init script

LANDS IN

`Azure Metrics`

`InsightsMetrics (with agent)`

Agent-based collection needs a data collection rule.

USE IT FOR

- Right-sizing clusters
- Spotting memory pressure and spill
- Capacity trends

Gap: Not collected into Log Analytics by default; most teams use the Databricks UI.

04 Spark application logs

Driver logs, stages, custom metrics

ROUTE

Cluster log delivery or custom log shipping

Driver and executor logs can be delivered to a storage location. Sending them to Log Analytics needs an init script or library that forwards Log4j output, or OpenTelemetry.

Set up in: Cluster config → Logging; init script

LANDS IN

`Custom tables (e.g. SparkLoggingEvent_CL)`

Custom tables are named by you and cost like any ingestion.

USE IT FOR

- Application errors across many jobs
- Custom business metrics from pipelines
- Stage-level performance analysis

Gap: You build and maintain the forwarding; check it works on each runtime version.

05 System tables Billing, job runs, queries

ROUTE

Stays in Databricks (Unity Catalog)

Usage, job run timelines, query history and cluster events are recorded in system tables and queried with SQL. They are not sent to Azure Monitor.

Set up in: Account admin grants access to system schemas

LANDS IN

- `system.billing.usage`
- `system.lakeflow.job_run_timeline`
- `system.query.history`

Export to Log Analytics only if you must, with a scheduled job.

USE IT FOR

- DBU cost by team, job and cluster
- Job duration and SLA trends
- Slow query analysis

Gap: Not real time, and outside Azure Monitor alerting. Use Databricks SQL alerts.

EVERY SIGNAL **Collect** · diagnostic setting → **Store** · Log Analytics → **Query** · KQL → **Act** · alerts, workbooks

03 Azure Monitor: the building blocks

COLLECT

Diagnostic settings

Per-resource rule that sends chosen log categories and metrics to one or more destinations.

`Databricks:` the main route for Databricks logs.

COLLECT

Activity log

Subscription-level record of every create, update and delete on Azure resources.

`Databricks:` changes to the workspace resource.

COLLECT

Azure Monitor Agent + DCR

An agent on VMs, configured by a data collection rule, that gathers logs and performance counters.

`Databricks:` optional, installed on cluster nodes with an init script.

STORE

Log Analytics workspace

Stores logs in tables, queried with KQL. The centre of Azure Monitor Logs.

`Databricks:` one table per Databricks log category.

STORE

Metrics

Numeric time series with 93 days of retention, fast to chart and alert on.

`Databricks:` limited; cluster VMs are short-lived.

ACT

Alerts + action groups

Rules on logs or metrics that notify people or trigger automation.

`Databricks:` job failures, permission changes, missing runs.

VISUALISE

Workbooks and dashboards

Interactive reports built from KQL queries; also Grafana and Power BI.

`Databricks:` a platform health workbook per environment.

SECURE

Microsoft Sentinel

SIEM built on Log Analytics. Correlates events across sources into incidents.

`Databricks:` analytics rules on Databricks audit tables.

SET UP THE DIAGNOSTIC SETTING

One per workspace. Requires the Premium plan.

1. Create a Log Analytics workspace in the same region, owned by operations.
2. Open the Databricks workspace resource → Monitoring → Diagnostic settings → Add.
3. Select the log categories you need; allLogs sends everything.
4. Choose destinations: Log Analytics for queries, Storage for cheap archive, Event Hubs for an external SIEM.
5. Turn on verbose audit logs in the workspace settings if you need command text.

COMMON CATEGORIES → LOG ANALYTICS TABLE

jobs → DatabricksJobs Runs started, succeeded, failed	**clusters → DatabricksClusters** Create, start, resize, terminate
notebook → DatabricksNotebook Attach, run commands (verbose)	**unityCatalog → DatabricksUnityCatalog** Table access, grants
accounts → DatabricksAccounts Sign-ins, tokens, admin changes	**secrets → DatabricksSecrets** Secret reads and scope changes
databrickssql → DatabricksDatabricksSQL Warehouse and query events	**workspace → DatabricksWorkspace** Workspace settings changes

AS CODE · TERRAFORM

Part of the workspace baseline, so no workspace exists without it.

```
resource "azurerm_monitor_diagnostic_setting" "dbw" {
  name                       = "diag-to-law"
  target_resource_id         = azurerm_databricks_workspace.prod.id
  log_analytics_workspace_id = azurerm_log_analytics_workspace.ops.id

  enabled_log {
    category_group = "allLogs"
  }
}
```

04 Log Analytics: store, retain, query

A **Log Analytics workspace** is an Azure resource that stores log data in tables. Each Databricks log category becomes its own table. You query them with **KQL** (Kusto Query Language), and pay mainly for the volume ingested and how long it is kept.

WORKSPACE DESIGN

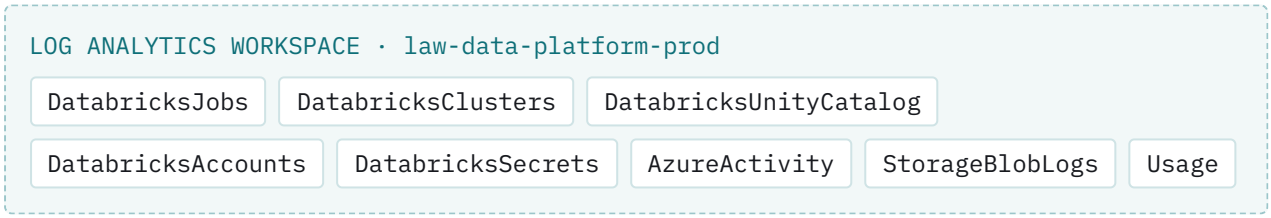
> **Central, per environment:** one workspace for prod and one for non-prod, shared by all data platform resources.

> **Same region** as the Databricks workspace to avoid cross-region transfer and residency questions.

> **Access:** Log Analytics Reader for the platform team; resource-context access lets a team see only its own resources.

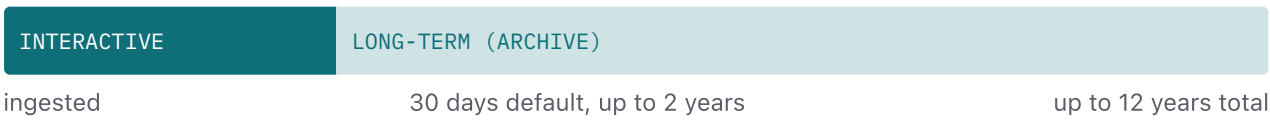
> **Retention per table:** keep audit tables for years, noisy diagnostic tables for weeks.

> **Table plans:** Analytics for data you alert on; cheaper Basic or Auxiliary plans for high-volume logs you rarely query.



RETENTION AND COST

Ingestion	Priced per GB ingested. Commitment tiers lower the rate at steady volume.
Interactive	Queryable at full speed. 30 days by default on Analytics tables, extendable up to 2 years.
Long-term	Cheaper retention up to 12 years in total. Query with search jobs or restore when needed.



Verbose audit logs and chatty categories drive most of the cost. Check the `Usage` table monthly and set a daily cap on non-production workspaces.

ANATOMY OF A ROW · DatabricksJobs

TimeGenerated 2026-09-24 02:14:07 When the event occurred	**ActionName** runFailed What happened	**Identity** {"email":"sp-etl-prod"} Who, as JSON
RequestParams {"jobId":"4821","runId":"99017"} Details, as JSON	**SourceIPAddress** 10.20.4.11 From where	**Response** {"statusCode":200} Result of the request

KQL by example

Each line of the query is one step in a pipeline.

01 Failed jobs in the last day

Which job runs failed, and which identity ran them?

`DatabricksJobs`	Start from the table the jobs category writes to.
`| where TimeGenerated > ago(1d)`	Always filter on time first: it is the cheapest filter.
`| where ActionName == "runFailed"`	Keep only failed runs.
`| extend jobId = tostring(` `  parse_json(RequestParams).jobId)`	RequestParams is JSON; pull out the field you need.
`| summarize failures = count()` `  by jobId, Identity`	Count failures per job and run-as identity.
`| order by failures desc`	Worst first.

Turn it into: a Sev 1 log search alert for production jobs.

02 Who creates clusters

Which users create or edit clusters most often? Useful for policy and cost conversations.

`DatabricksClusters`	Cluster lifecycle events.
`| where TimeGenerated > ago(7d)`	Last week.
`| where ActionName in ("create", "edit")`	Creation and configuration changes only.
`| extend who = tostring(` `  parse_json(Identity).email)`	Identity is JSON too.
`| summarize actions = count() by who, ActionName`	Group by person and action.

Turn it into: a weekly workbook tile for cost reviews.

03 Permission changes

Every Unity Catalog GRANT or REVOKE, with who made it and on what.

`DatabricksUnityCatalog`	Unity Catalog audit events.
`| where TimeGenerated > ago(1h)`	Match the alert's evaluation window.
`| where ActionName == "updatePermissions"`	Grants and revokes.
`| project TimeGenerated, Identity,` `  RequestParams, SourceIPAddress`	Keep only the columns a reviewer needs.

Turn it into: an alert to the security team, and a Sentinel analytics rule.

04 A job that stopped running

Failures are easy to catch. A job whose schedule was paused or deleted produces no events at all.

`DatabricksJobs`	Job events.
`| where TimeGenerated > ago(26h)`	Slightly longer than the daily schedule.
`| where ActionName == "runSucceeded"`	Successful completions.
`| where RequestParams has "nightly_sales"`	The job you expect.
`| count`	Alert when the count is 0.

Turn it into: an alert that fires when an expected event is missing.

05 What is costing us

Log Analytics bills on ingested gigabytes. The Usage table shows which tables are responsible.

`Usage`	Built-in table of ingestion volume.
`| where TimeGenerated > ago(30d)`	Last month.
`| where IsBillable == true`	Ignore free data types.
`| summarize GB = sum(Quantity) / 1000 by DataType`	Quantity is in MB.
`| order by GB desc`	Biggest tables first.

Turn it into: a monthly review, and a daily cap on non-production workspaces.

05 From query to action: alerts and dashboards

ANATOMY OF A LOG SEARCH ALERT

1 · SCOPE

Log Analytics workspace

law-data-platform-prod

2 · CONDITION

KQL + threshold

Failed runs of production jobs > 0, evaluated every 15 minutes over the last 15 minutes.

3 · SEVERITY

Sev 0–4

Sev 1 for production pipelines, Sev 3 for dev. Decides who is woken up.

4 · ACTION GROUP

Who and how

Email and Teams to the on-call team; webhook to ServiceNow; a Logic App for automatic fixes.

STARTER ALERT RULES

ALERT	TABLE	SEV	NOTIFY
Production job failed	`DatabricksJobs`	Sev 1	Data engineering on-call (Teams + phone)
Expected job did not run	`DatabricksJobs`	Sev 2	Data engineering on-call
Unity Catalog permission changed	`DatabricksUnityCatalog`	Sev 2	Security team; Sentinel incident
Admin role or token created	`DatabricksAccounts`	Sev 2	Security team
Workspace or network resource changed	`AzureActivity`	Sev 3	Platform team
Log ingestion above daily budget	`Usage`	Sev 4	Platform team (email)

Use Azure Monitor for

- → Real-time alerts on failures and security events
- → Correlation with storage, network, Key Vault and Entra ID logs
- → Long-term, tamper-resistant retention
- → Feeding Sentinel and ITSM tools

Use Databricks system tables for

- → DBU cost by workspace, team, job and cluster
- → Job duration and SLA trends over months
- → Query performance and warehouse sizing
- → Account-level events across all workspaces

Diagnostic settings on every workspace

Deploy them with the workspace, as code. A workspace without them is invisible to operations.

One central Log Analytics workspace

Per environment, owned by operations. Set retention to your audit requirement and watch ingestion cost.

Every alert has an owner

Start with a few high-value alerts routed to a named team. An alert nobody acts on trains people to ignore alerts.